

树与二叉树

层次关系怎样递归表示、遍历并保持结构不变量

408 · 数据结构 Topic DS-04

母问题：层次关系怎样变成可执行的访问顺序？

树的生成链是

Hierarchy → Recursive Representation → Frontier Traversal → Local Change → Structural Invariant.

节点的子树仍是树，所以递归定义自然产生递归算法；把尚未展开的节点放入栈或队列，就得到显式遍历。访问时机决定先序、中序、后序，取出规则决定层序。Huffman 则把“树”用于编码优化：反复合并最小权值，目标是带权路径长度。

本册定位与 Owner 边界

- **Owns:** 树/森林与二叉树的性质和存储、孩子兄弟转换、递归与迭代遍历、线索化、高度、Huffman 主干。
- **Uses:** DS02 的栈/队列端点契约；DS05 的优先队列接口。
- **Stops:** 堆内部上滤下滤、BST/AVL/B/B+ 查找与旋转、图遍历正确性归各自 Topic。

目录

1 对象与表示	2
2 遍历：访问时机与 frontier	2
3 树、森林与二叉树：孩子兄弟表示是可逆接口	2
4 线索二叉树：把空指针改造成遍历序列邻接关系	2
4.1 中序线索化的状态机	2
5 Huffman：树是合并结果，不是搜索树	3
6 代码：从节点所有权到访问顺序	3
6.1 节点与递归遍历	3
6.2 显式栈与层序队列	4
6.3 Huffman 合并调用	5
6.4 测试证据	6
7 复杂度与概念边界	7
8 做题控制协议与压缩复原	7

1 对象与表示

二叉树节点至少包含值、左子树、右子树。子树为空是合法状态，不是异常；叶节点的两个孩子都为空。用唯一所有权表示时，父节点拥有子节点，销毁根即可递归释放整棵树。树的结构不变量是：每个节点至多有一个父关系入口，沿孩子指针不会回到祖先，且遍历所得节点数等于可达节点数。

同一棵树的四种读法

对根 1、左子树 2(4,5)、右子树 3，先序为 1,2,4,5,3，中序为 4,2,5,1,3，后序为 4,5,2,3,1，层序为 1,2,3,4,5。树没有变化，改变的是“何时从 frontier 取出并处理根”。

2 遍历：访问时机与 frontier

递归先序在进入节点时处理；中序在左子树返回后处理；后序在两个子树都返回后处理。显式栈保存尚未完成的调用现场：先序压入右子树再压左子树，才能保证左子树先出栈。中序先沿左链压栈，弹出一个节点后转向右子树。层序使用 FIFO 队列，按深度展开；其队列语义来自 DS02，树的覆盖正确性由本册维护。

3 树、森林与二叉树：孩子兄弟表示是可逆接口

一般树的节点可以有任意多个孩子。孩子兄弟表示只保留两个方向：

$\text{firstChild} = \text{第一个孩子}, \quad \text{nextSibling} = \text{下一个兄弟}.$

把 firstChild 解释为二叉树左孩子、 nextSibling 解释为右孩子，就得到树到二叉树的转换。森林则把各棵树的根按兄弟链连接：第一棵树根是二叉树根，后续树根依次位于右链。

该转换保持层次与兄弟次序，不保持“二叉树左右孩子”的原语义。由指针解释可推出遍历对应：

树的先根遍历 = 对应二叉树的先序遍历, 树的后根遍历 = 对应二叉树的中序遍历.

森林遍历还要沿根的兄弟链完成每棵树，不能只处理第一棵。

4 线索二叉树：把空指针改造成遍历序列邻接关系

含 n 个节点的二叉链表有 $2n$ 个孩子指针，而非空边只有 $n-1$ 条，因此有 $n+1$ 个空指针。线索化利用这些空槽保存某种遍历次序下的前驱或后继。因为字段可能表示真实孩子或线索，必须增加标志：

$\text{ltag} = 0 \Rightarrow \text{lchild}$ 是左孩子, $\text{ltag} = 1 \Rightarrow \text{lchild}$ 是前驱线索,

右侧同理。忽略 tag 沿线索递归，会把序列关系误当结构边，造成重复访问甚至循环。

4.1 中序线索化的状态机

维护 pre 指向刚刚访问的节点，在访问当前节点 p 时：

1. 若 p 的左孩子为空，令 $p.\text{lchild} = pre$ 且 $p.\text{ltag} = 1$ ；
2. 若 pre 存在且其右孩子为空，令 $pre.\text{rchild} = p$ 且 $pre.\text{rtag} = 1$ ；
3. 更新 $pre = p$ ，再处理右子树。

这不是任意补指针，而是在同步构造中序序列相邻节点的关系。线索化完成后找中序后继：若 $rtag = 1$ ，线索直接给出后继；若 $rtag = 0$ ，后继是右子树中沿真实左孩子到达的最左节点。

线索树边界

线索只服务于指定遍历次序；中序前驱/后继不能直接当作先序关系。某些先序、后序邻接仍可能需要父指针或重新定位，不能把“有线索”理解成所有遍历操作都严格 $O(1)$ 。

5 Huffman：树是合并结果，不是搜索树

给定符号权重，带权路径长度为 $\sum_i w_i d_i$ 。每次合并两个最小权值 x, y ，把 $x + y$ 重新放回候选集合，累计代价 $x + y$ 。该贪心过程依赖最小优先队列；本册只固定合并契约与代价，优先队列内部实现由 DS05 Own。单个符号的编码长度约定为 0，空输入代价为 0。

6 代码：从节点所有权到访问顺序

完整实现位于 `code/ds04_tree_binary.hpp`，测试位于 `code/ds04_tree_binary_test.cpp`；代码块直接引用真实源文件。

6.1 节点与递归遍历

Listing 1: 唯一所有权节点与递归先中后序

```

12
13 struct Node {
14     int value;
15     std::unique_ptr<Node> left;
16     std::unique_ptr<Node> right;
17 };
18
19 inline std::unique_ptr<Node> leaf(int value) {
20     return std::make_unique<Node>(Node{value, nullptr, nullptr});
21 }
22
23 inline void preorder_recursive(const Node* root, std::vector<int>& output)
24 {
25     if (root == nullptr) {
26         return;
27     }
28     output.push_back(root->value);
29     preorder_recursive(root->left.get(), output);
30     preorder_recursive(root->right.get(), output);
31 }
32
33 inline void inorder_recursive(const Node* root, std::vector<int>& output)
34 {
35     if (root == nullptr) {
36         return;
37     }
38     inorder_recursive(root->left.get(), output);
39     output.push_back(root->value);
40     inorder_recursive(root->right.get(), output);
41 }
42
43 inline void postorder_recursive(const Node* root, std::vector<int>& output)
44 {
45     if (root == nullptr) {
46         return;
47     }
48     postorder_recursive(root->left.get(), output);
49     postorder_recursive(root->right.get(), output);
50     output.push_back(root->value);
51 }

```

```

36     inorder_recursive(root->left.get(), output);
37     output.push_back(root->value);
38     inorder_recursive(root->right.get(), output);
39 }
40
41 inline void postorder_recursive(const Node* root, std::vector<int>& output
    ) {
42     if (root == nullptr) {
43         return;

```

递归基是空指针立即返回；每个非空节点只处理一次，子树调用返回后再按访问时机写入输出。

6.2 显式栈与层序队列

Listing 2: 迭代遍历与层序 frontier

```

45     postorder_recursive(root->left.get(), output);
46     postorder_recursive(root->right.get(), output);
47     output.push_back(root->value);
48 }
49
50 inline std::vector<int> preorder_iterative(const Node* root) {
51     std::vector<int> output;
52     if (root == nullptr) {
53         return output;
54     }
55     std::stack<const Node*> pending;
56     pending.push(root);
57     while (!pending.empty()) {
58         const Node* current = pending.top();
59         pending.pop();
60         output.push_back(current->value);
61         if (current->right != nullptr) {
62             pending.push(current->right.get());
63         }
64         if (current->left != nullptr) {
65             pending.push(current->left.get());
66         }
67     }
68     return output;
69 }
70
71 inline std::vector<int> inorder_iterative(const Node* root) {
72     std::vector<int> output;
73     std::stack<const Node*> pending;
74     const Node* current = root;
75     while (current != nullptr || !pending.empty()) {
76         while (current != nullptr) {

```

```

77     pending.push(current);
78     current = current->left.get();
79 }
80 current = pending.top();
81 pending.pop();
82 output.push_back(current->value);
83 current = current->right.get();
84 }
85 return output;
86 }
87
88 inline std::vector<int> level_order(const Node* root) {
89     std::vector<int> output;
90     if (root == nullptr) {
91         return output;
92     }
93     std::queue<const Node*> pending;
94     pending.push(root);
95     while (!pending.empty()) {
96         const Node* current = pending.front();
97         pending.pop();
98         output.push_back(current->value);
99         if (current->left != nullptr) {

```

显式结构保存的不是“所有节点”，而是尚未展开的边界；栈的压入顺序和队列的端点顺序直接生成输出次序。

6.3 Huffman 合并调用

Listing 3: 最小权值合并与累计代价

```

111     return 0;
112 }
113 return 1 + std::max(height(root->left.get()), height(root->right.get()))
114 );
115 }
116
117 inline int huffman_weighted_path_length(const std::vector<int>& weights) {
118     if (weights.size() < 2) {
119         return 0;
120     }
121     std::priority_queue<int, std::vector<int>, std::greater<int>> pending(
122         weights.begin(), weights.end());
123     int total = 0;
124     while (pending.size() > 1) {
125         const int first = pending.top();
126         pending.pop();
127         const int second = pending.top();

```

```
127     pending.pop();
128     const int merged = first + second;
129     total += merged;
130     pending.push(merged);
```

6.4 测试证据

Listing 4: 空树、遍历覆盖与 Huffman 边界测试

```
17 std::unique_ptr<Node> sample_tree() {
18     auto root = leaf(1);
19     root->left = leaf(2);
20     root->right = leaf(3);
21     root->left->left = leaf(4);
22     root->left->right = leaf(5);
23     return root;
24 }
25
26 void test_empty_and_singleton() {
27     std::vector<int> output;
28     preorder_recursive(nullptr, output);
29     assert(output.empty());
30     assert(level_order(nullptr).empty());
31     auto root = leaf(9);
32     assert(height(root.get()) == 1);
33     assert((preorder_iterative(root.get()) == std::vector<int>{9}));
34     assert((inorder_iterative(root.get()) == std::vector<int>{9}));
35 }
36
37 void test_traversal_orders() {
38     auto root = sample_tree();
39     std::vector<int> preorder;
40     std::vector<int> inorder;
41     std::vector<int> postorder;
42     preorder_recursive(root.get(), preorder);
43     inorder_recursive(root.get(), inorder);
44     postorder_recursive(root.get(), postorder);
45     assert((preorder == std::vector<int>{1, 2, 4, 5, 3}));
46     assert((preorder_iterative(root.get()) == preorder));
47     assert((inorder == std::vector<int>{4, 2, 5, 1, 3}));
48     assert((inorder_iterative(root.get()) == inorder));
49     assert((postorder == std::vector<int>{4, 5, 2, 3, 1}));
50     assert((level_order(root.get()) == std::vector<int>{1, 2, 3, 4, 5}));
51     assert(height(root.get()) == 3);
52 }
```

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \\\
```

```
code/ds04_tree_binary_test.cpp -o /tmp/ds04_test
/tmp/ds04_test
```

7 复杂度与概念边界

操作	递归/迭代遍历	额外空间	前提	边界
遍历 n 节点	$O(n)$	$O(h)$ 或 $O(w)$	每节点一次	空树返回空序列
高度	$O(n)$	$O(h)$	子树高度定义固定	空树高度约定
Huffman 合并	$O(k \log k)$	$O(k)$	最小优先队列	$k < 2$ 代价为 0

概念 A	≠ 概念 B	判据	错误后果
树节点所有权	≠ 遍历顺序	生命周期 vs 访问时机	泄漏或次序错
层序遍历	≠ BFS 图遍历	树无环且无需一般 visited	把图结论硬套树
Huffman 树	≠ BST	最小带权路径 vs 有序查找	混淆目标

8 做题控制协议与压缩复原

先画根、子树范围和空孩子；再选访问时机或 frontier；每次节点只入队/入栈一次并验证覆盖；若题目问 Huffman，改写为“合并两个最小权值并累计”。忘掉口诀后复原五问：节点拥有谁？未展开边界在哪里？当前节点何时处理？空子树如何结束？目标是访问次序还是编码代价？

全科交接

DS04 从 DS02 接收 LIFO/FIFO frontier，从 DS05 接收最小优先队列接口；向 DS-B01 提供树遍历实例，向 DS09 提供“树结构可承载有序索引”的前置直觉，但不拥有 BST/AVL 机制。

旧总册关于递归层次、遍历访问时机与 Huffman 合并的内容已吸收；树/森林转换与线索化已按考纲补入本册，遍历序列重建和更复杂编码变体保留为扩展，不能覆盖本册主干。